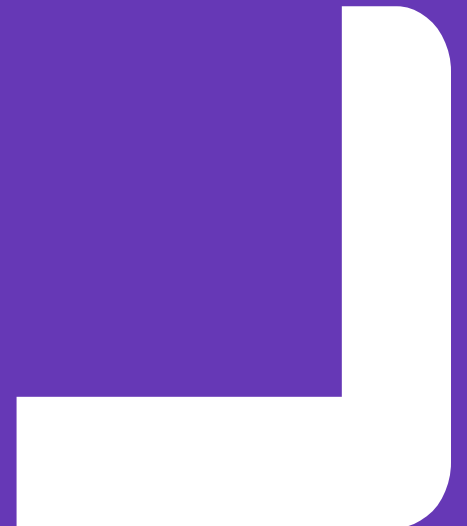


# SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens  
[christoph.andriessens@hs-weingarten.de](mailto:christoph.andriessens@hs-weingarten.de)



# 「 EINFÜHRUNG IN VERSIONSVERWALTUNGSSYSTEME



# Themen in dieser Vorlesung



# Lernziele

- Versionsmanagement als Werkzeug für die Software-Entwicklung kennen lernen
- Notwendige Grundbegriffe lernen



# Karin

Karin (siehe Abbildung 9.1) ist für ihre Software-Entwicklung auf der Suche nach einem geeigneten Versionsmanagementsystem (VMS): Wie kann das gesuchte VMS die Teammitglieder bei der anspruchsvollen Organisation der Programmdateien behilflich sein - schließlich entwickeln Sie drei Produkte gleichzeitig; da kann man schnell den Überblick verlieren? Das VMS sollte die unterschiedlichen Lokationen der Teammitglieder berücksichtigen und verteiltes Arbeiten unterstützen (und nicht behindern)! Kann das VMS Karin und den Teammitgliedern ein Gefühl der Sicherheit geben - Ja, wir haben alles unter Kontrolle.



Abbildung 9.1: Karin ist 25 Jahre alt, sie entwickelt als Mitbegründerin ein Startup drei Produkte im Bereich Social Media. Die Teams sind verteilt über Standorte in Europa und USA.

# Motivation – Backup



Karin arbeitet im Home-Office am Produkt für die Anbindung an die Twitter-Plattform – nach 6h Nachtschicht gelingt ihr der erfolgreiche Durchbruch. Müde – aber zufrieden – geht Sie zum Schlafen.

Am anderen Morgen muss Sie feststellen, dass ihr Notebook nicht mehr startet.

Die gesamte Arbeit der letzten 6h war vergebens.

Was würde Sie dafür geben, wenn Sie ein Backup ihrer Arbeit vom Vortag erstellt hätte!

# Motivation – Organisation



Karin arbeitet an 3 Produkten gleichzeitig. Deswegen muss Sie häufig zwischen verschiedenen Arbeiten hin- und herwechseln. Dieser fortwährende Wechsel stresst Karin immer mehr, da Sie sich ständig wieder in neue Gegebenheiten und Anforderungen des jeweiligen Produkts einarbeiten muss. Dies erfordert von ihr große Disziplin.

Gerade beim Entwickeln des jeweiligen Produkts muss Sie ständig zwischen verschiedenen Versionen unterschiedlicher Programmdateien hin- und herwechseln. Sie muss sich merken, wo Sie die Dateien abgelegt hat; was Sie dort als letztes gemacht hat; was noch zu tun ist; ...

# Motivation – Kollaboration



Karin arbeitet an 3 Produkten gleichzeitig. Da Sie die Produkte alleine nicht fertigstellen kann, helfen ihr insgesamt 3 Teams mit 30 Personen. Diese Teammitglieder sind verteilt an verschiedenen Standorten über Europa und den USA.

Wie kann Sie nur die Arbeiten von so vielen Personen, die auch noch zu unterschiedlichen Zeiten tätig sind, koordinieren!



# Motivation

Lösung: Versionsverwaltungssysteme

Backup

Organisation

Kollaboration

# Versionsverwaltung

Versionsverwaltung ist der Prozess, unterschiedliche Versionen von Softwarekomponenten oder anderen Systemressourcen und die Systeme (mit ihren Versionen), die diese Komponenten verwenden, zu verfolgen.

**Beinhaltet:** Gleichzeitige Arbeit mehrerer Entwickler auf denselben Komponenten zu ermöglichen, ohne dass diese sich gegenseitig behindern.

**Beispiele** für bekannte Werkzeuge für Versionsverwaltung:

- Concurrent Versions System (CVS, älteres System)
- Subversion (SVN)
- Git

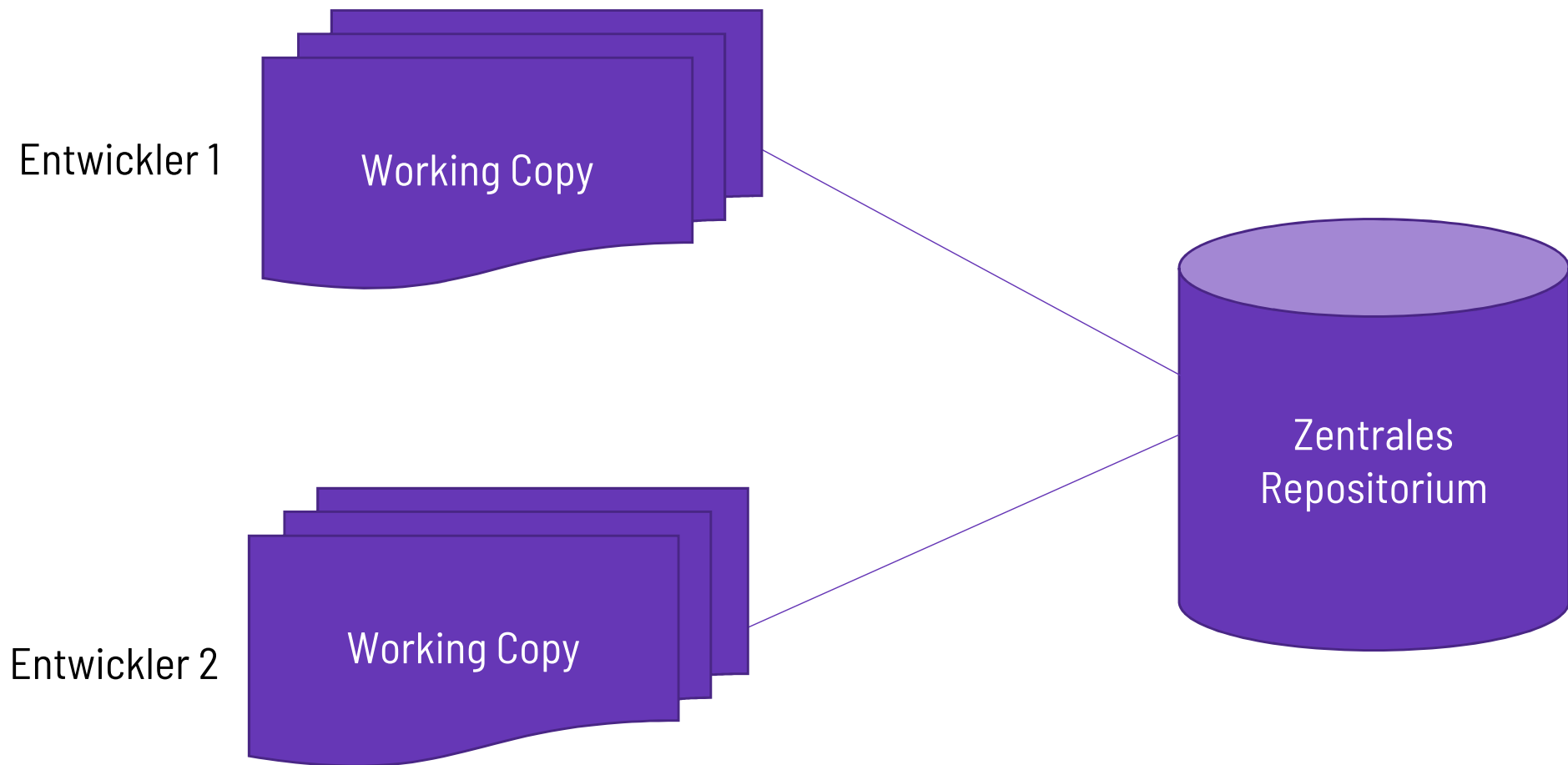
# Zentrale Eigenschaften von Versionsverwaltungssystemen

- Management der Ablage der Artefakte in einem **Repository**
- Eindeutige **Identifizierung** von Versionen und Releases
- Aufzeichnung einer **Veränderungshistorie**
- Unterstützung für **gleichzeitige Entwicklungsarbeiten** mehrerer Personen unabhängig von einander
- Unterstützung des **Entwicklungsprojektes**

# Zwei Arten von Versionsverwaltungssystemen

- **Zentrale Systeme:** Ein einzelnes Hauptrepository enthält alle Versionen der Softwarekomponenten, die in Entwicklung sind.
- **Verteilte Systeme:** Mehrere Versionen des Repositoriums existieren zur gleichen Zeit an unterschiedlichen Orten.

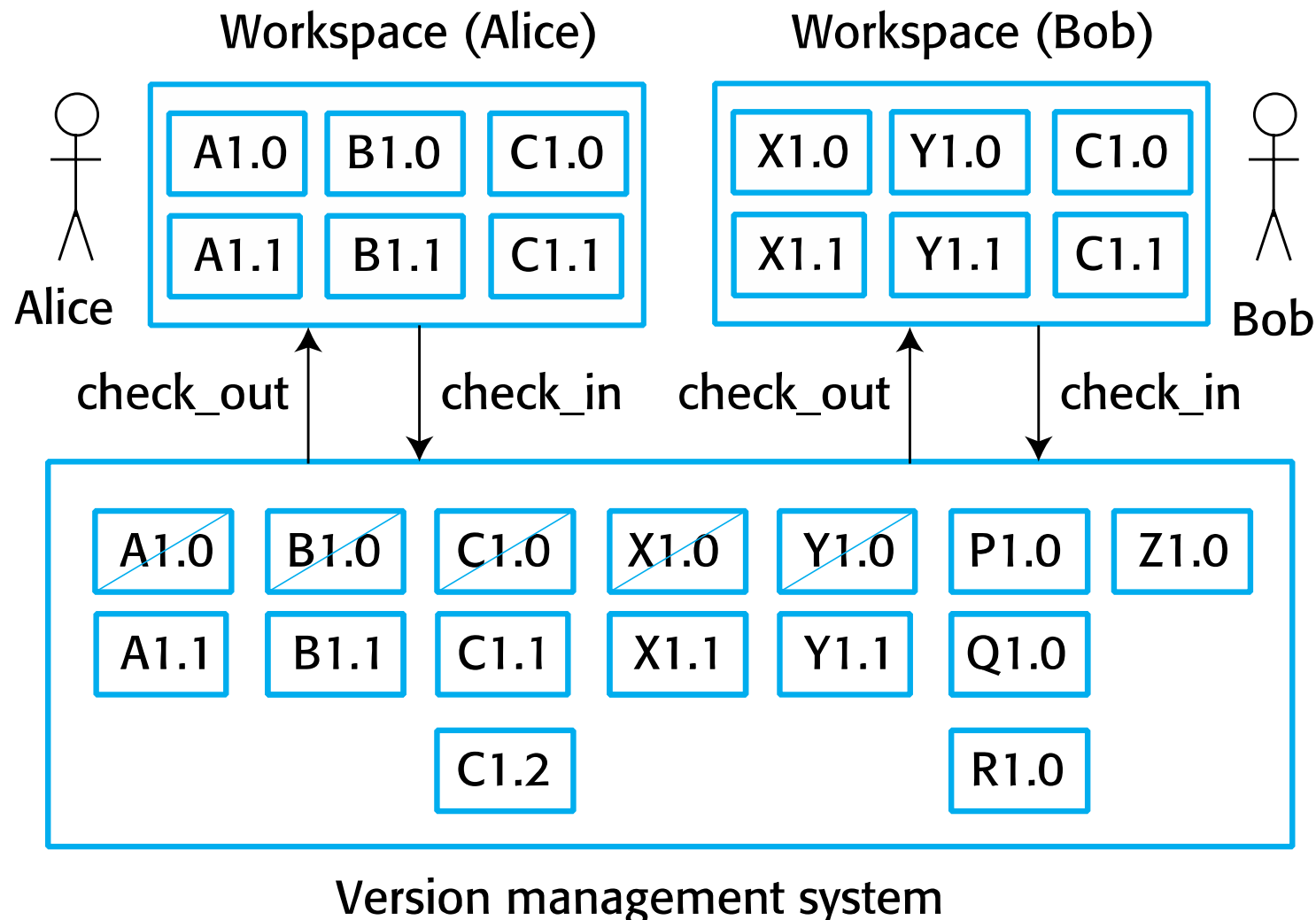
# Struktur zentraler Versionsverwaltungssysteme



# Zentrale Versionsverwaltungssysteme

- Entwickler kopieren Dateien aus den zentralen Repository in ihren privaten Arbeitsbereich („Check-Out“)
- Wenn sie fertig sind, kopieren sie die veränderten Dateien wieder zurück in das zentrale Repository („Check-In“ oder „commit“)
  - Ein Commit zeichnet die Änderungen der Dateien im Repository auf
- Wenn mehrere Personen auf derselben Datei arbeiten, registriert das Repository die Check-Outs und
  - verhindert entweder grundsätzlich solche Check-Outs
  - oder führt die Arbeiten anlässlich des Commits nach unterschiedlichen Strategien zusammen

# Zentrale Versionskontrollsysteme: Check-Out und Check-In



# Zentrale Versionsverwaltungssysteme

Beispielsweise SVN (Subversion):

- Zentraler Server, auf dem alle Versionsstände abgelegt sind
- Clients holen immer nur eine Working Copy, auf der lokal gearbeitet wird
- Jeder Commit geht direkt auf den Server

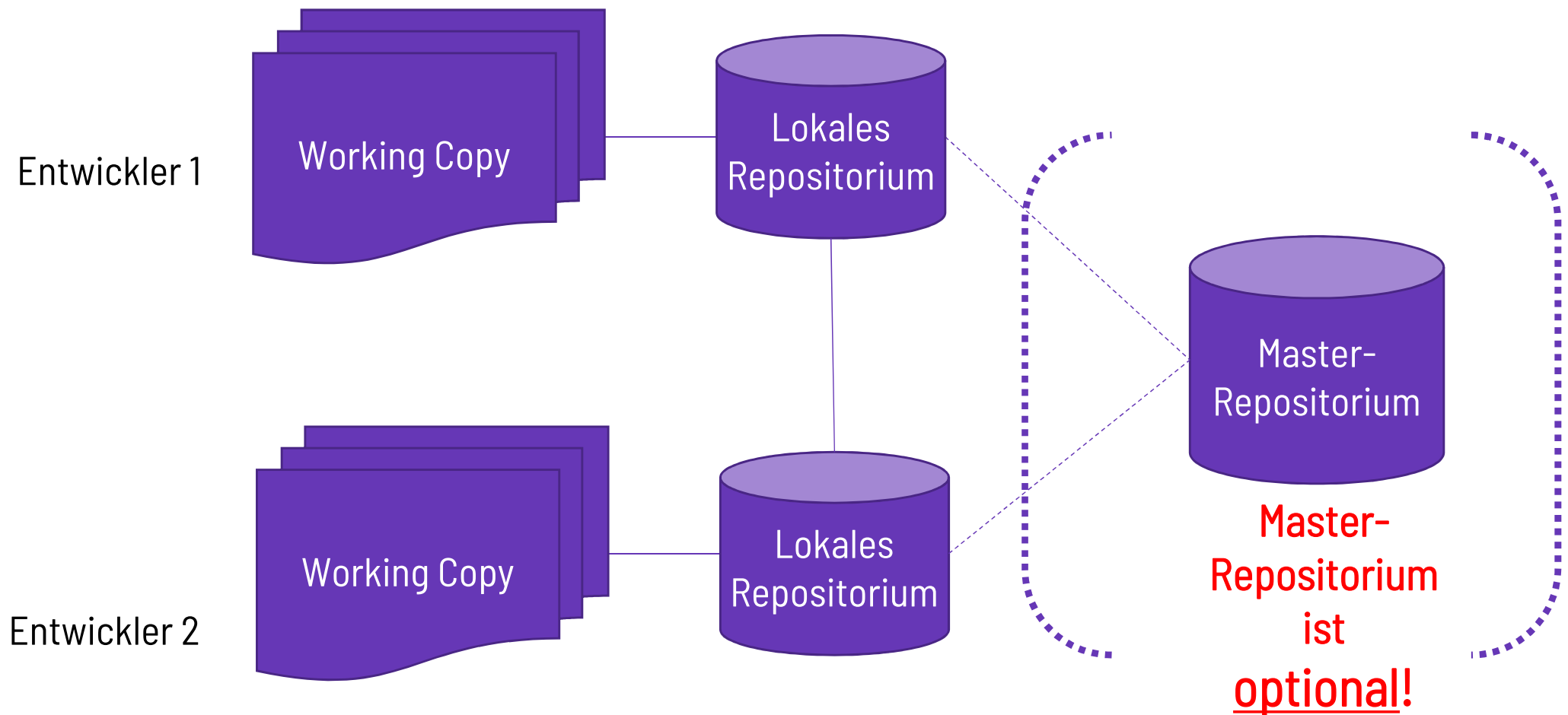




# Verteilte Versionsverwaltungssysteme

- Auf separatem Server wird ein **Master-Repository** erzeugt, das den Code des Gesamtteams enthält
- Anstatt Check-Out von Dateien aus Master-Repository: Jeder Entwickler lädt Klon des Master-Repositories als **lokales Repository** auf eigenen PC
- Die Entwickler arbeiten jeweils mit ihrem lokalem Repository, als ob es ein zentrales wäre (lokale Check-Outs, lokale Commits)
- Zu gewissen Zeitpunkten geben sie ihre Veränderungen an ein anderes Repository (oft auch ein sog. Master-Repository) weiter („Push“)

# Struktur verteilter Versionsverwaltungssysteme



# Verteilte Versionsverwaltungssysteme

Beispielsweise Git:

- Jeder Teilnehmer hat sein eigenes lokales Repository und eine zusätzliche Working Copy
- Commits gehen in das lokale Repository
- Das lokale Repository ist eine Kopie bzw Zusammenführung aller anderen Repositories
- Meistens gibt es ein zentrales Remote Repository als "Master-Repositoryum"

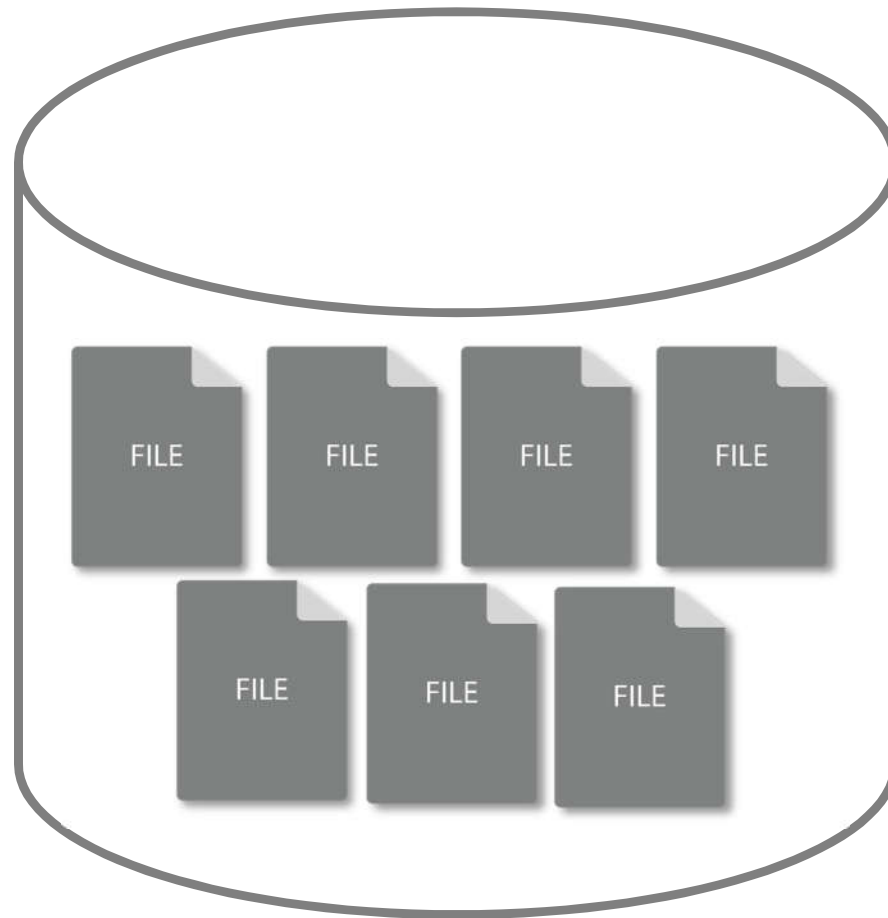


# Zentral und verteilte Systeme: Wichtige Unterschiede

- Performance
- Sperren von Dateien
- Offline-Arbeitsweise
- Speicherplatzverbrauch
- Redundanz

# Immer *eine* geordnete Folge von Änderungen?

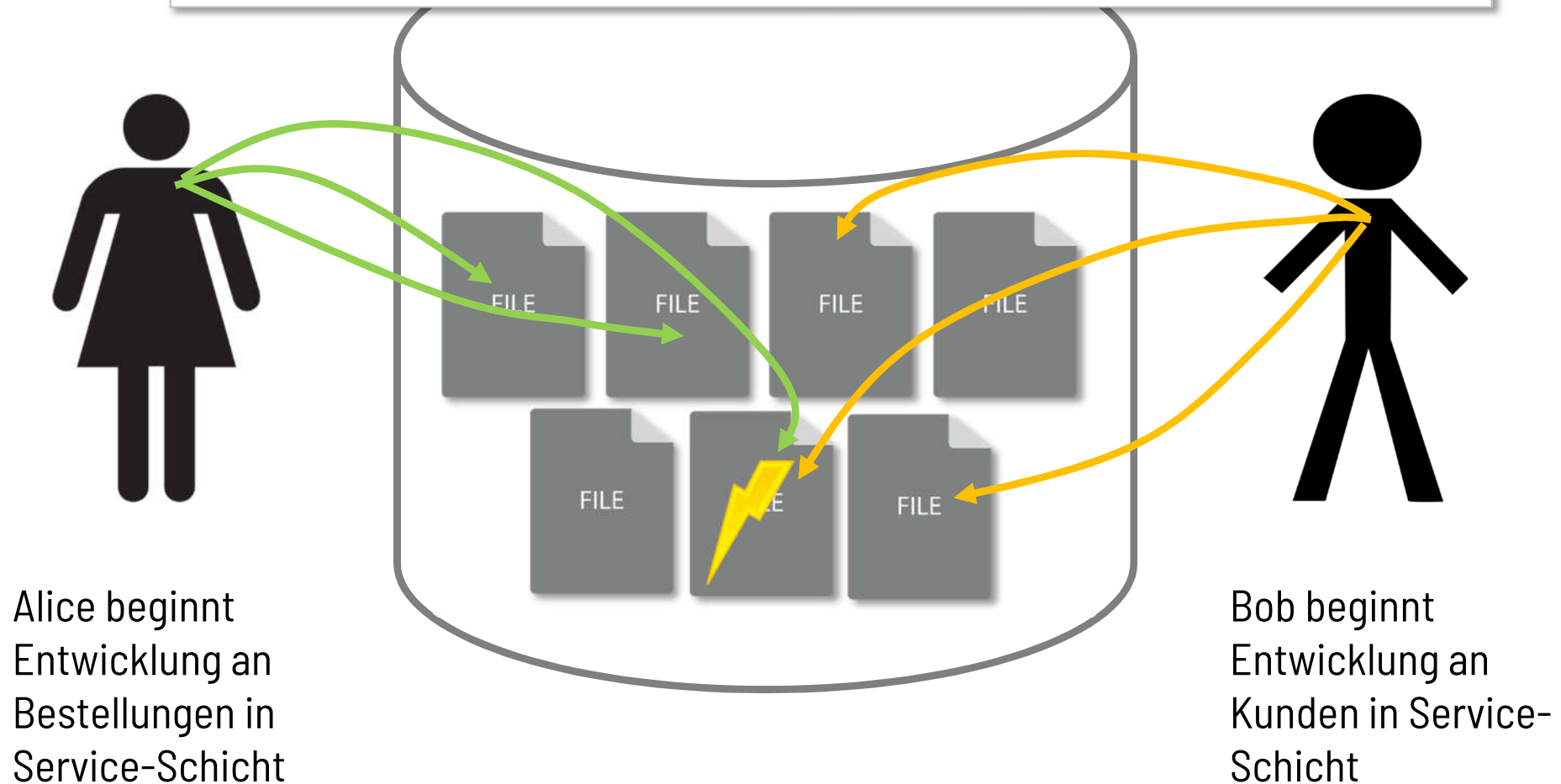
Lauffähiges, fehlerfreies System im Repository



# Immer *eine* geordnete Folge von Änderungen?

## Konsequenzen:

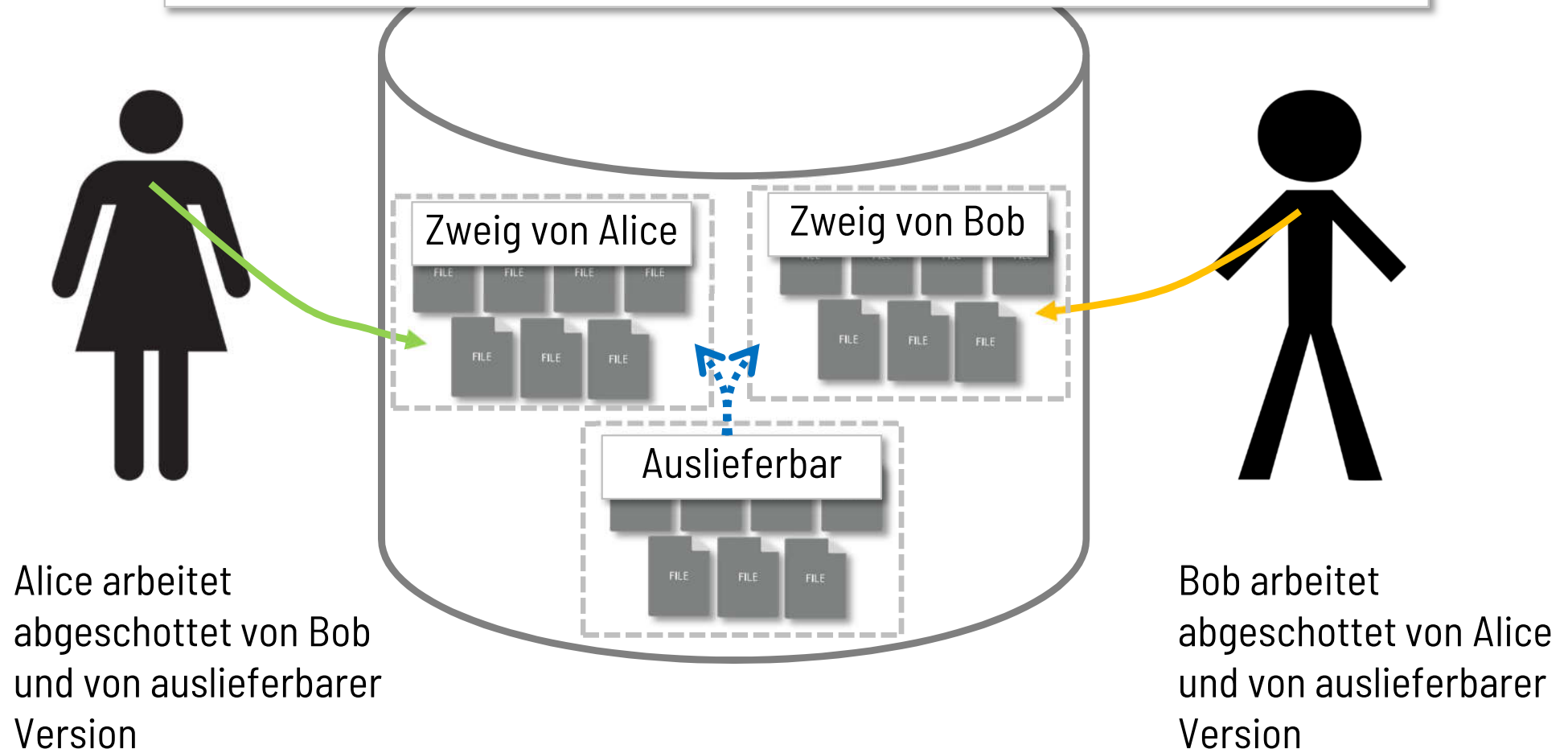
1. Aktueller Zustand des Systems während Entwicklung ist nicht mehr auslieferbar
2. Alice und Bob stören sich gegenseitig



# Immer *eine* geordnete Folge von Änderungen?

Es kann günstiger sein, mehrere Folgen von Änderungen in jeweils zusammenhängende Folgen zu trennen:

## ➤ Verzweigungen



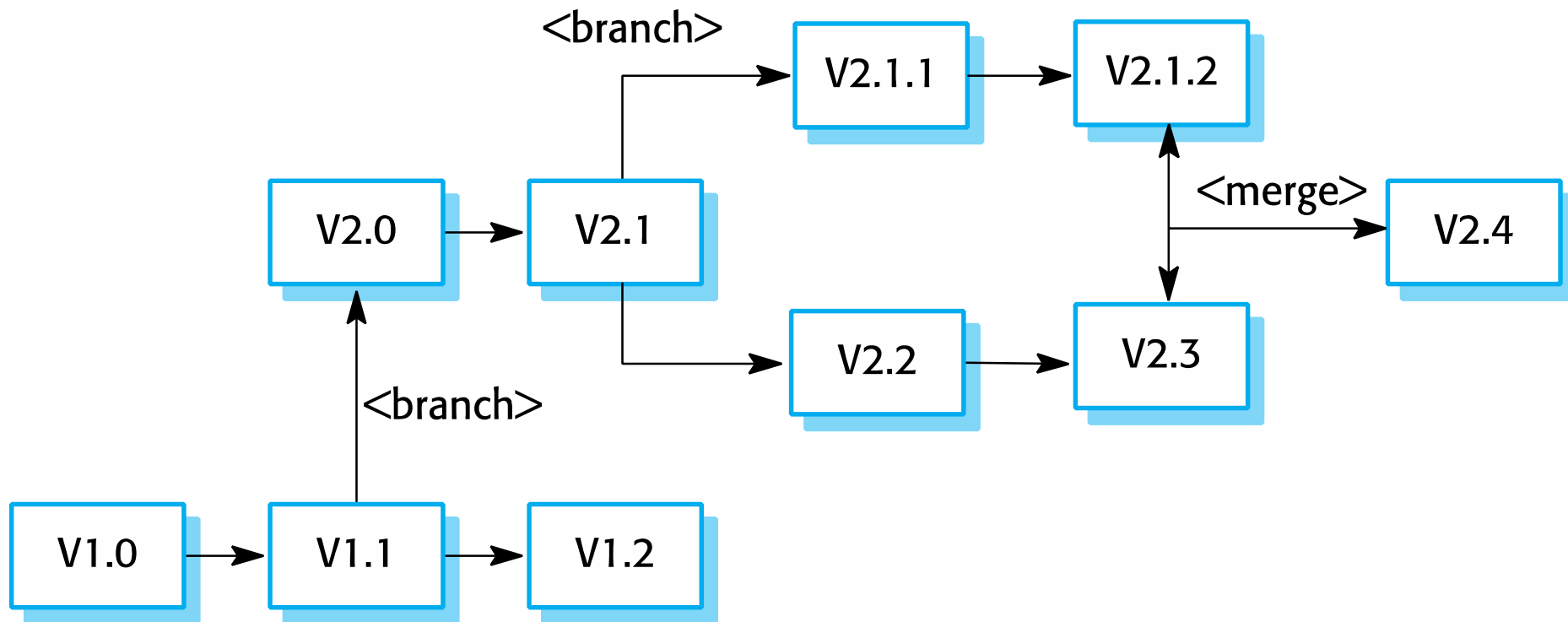
# Branching und Merging (Abzweigen und Verschmelzen)

Verschiedene Entwickler arbeiten parallel unabhängig voneinander auf mehreren Versionen des Quelltextes und verändern ihn auf verschiedenen Wegen:

- Sie arbeiten auf unterschiedlichen Zweigen („Branches“)
- Das Abspalten nennt man „Branching“
- Einsatz oft für Implementierung einzelner Features („Feature Branch“)
- Anstatt linearer Folge von Versionen einer Komponente: Es kann auch mehrere unabhängige Folgen von Versionen geben
- Das spätere Zusammenführen von unabhängigen Versionen einer Komponente zu einer Version heißt „Merging“ – dafür oft Konfliktauflösung nötig



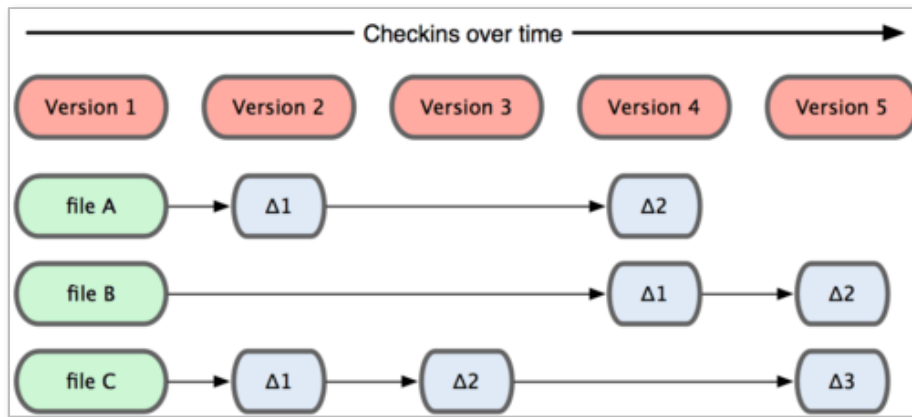
# Branching und Merging (Abzweigen und Verschmelzen)



Verschiedene Versionen einer Komponente mit Abzweigungen und einer Verschmelzung

# Versionen von Dateien verwalten

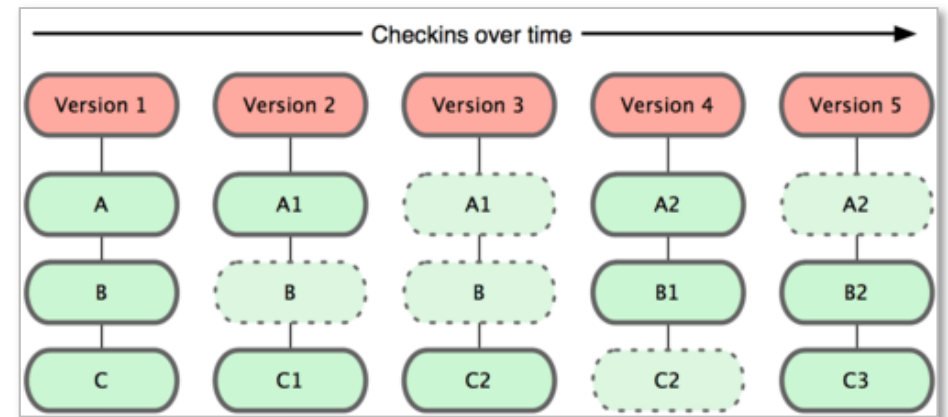
## Delta-Ansatz (etwa Subversion, CVS)



Nur **Unterschiede** zwischen Versionen einer Datei werden gespeichert

- Braucht wenig Speicherplatz
- Langsamer

## Snapshot-Ansatz (Git)



Bei jedem Commit wird der Gesamtzustand **aller Dateien** gespeichert („Snapshot“) – hat sich eine Datei nicht verändert, wird für diese auf vorherigen Zustand verwiesen

- Braucht mehr Speicherplatz
- Schneller

# Statistiken zu Googles Quelltexten

Erinnerung

Software aller Google-Anwendungen und Dienste

@scale-Konferenz 2015, Racc

Größtes Repository der Welt?

## Google repository statistics

As of Jan 2015

|                        |                    |
|------------------------|--------------------|
| Total number of files* | 1 billion          |
| Number of source files | 9 million          |
| Lines of code          | 2 billion          |
| Depth of history       | 35 million commits |
| Size of content        | 86 terabytes       |
| Commits per workday    | 45 thousand        |

\*The total number of files includes source files copied into release branches, files that are deleted at the latest revision, configuration files, documentation, and supporting data files.

# Google repository Januar 2015

Erinnerung

## Google repository usage

### Human users

- 25 thousand Googlers in dozens of offices around the world

### On an average workday

- 15 thousand commits by humans
- 30 thousand commits by automated systems
- Billions of file read requests\* (800K QPS at daily peak)

\*Google recently open-sourced a subset of the internal build system, see [www.bazel.io](http://www.bazel.io)